

Reprezentacje kodu źródłowego na potrzeby modeli i algorytmów głębokiego uczenia

1st inż. Jędrzej Dubiak

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178878@pw.edu.pl

2nd inż. Rafał Pytko

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178878@pw.edu.pl

3rd inż. Michał Jagodziński

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178878@pw.edu.pl

4th inż. Bartosz Dybowski

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178878@pw.edu.pl

Streszczenie—TODO na koniec

Słowa kluczowe—uczenie głębokie, reprezentacja kodu źródłowego

I. WSTĘP

Ostatnia dekada przyniosła ogromny postęp w dziedzinie uczenia maszynowego, szczególnie dzięki metodom uczenia głębokiego (Deep Learning), które zrewolucjonizowały obszary takie jak rozpoznawanie obrazów czy przetwarzanie języka naturalnego. Obecnie metody te są coraz szerzej adaptowane w inżynierii oprogramowania, gdzie wspomagają rozumienie i analizę programów komputerowych. Fundamentem sukcesu uczenia głębokiego w tym obszarze jest uczenie reprezentacji (representation learning), czyli proces przekształcania surowych danych w format, który modele mogą efektywnie przetwarzać [1].

Reprezentacja kodu źródłowego to proces transformacji tekstowej postaci kodu programu w ustrukturyzowany, ogólny format wejściowy akceptowalny dla modeli uczenia głębokiego. Modele te nie są w stanie bezpośrednio zrozumieć surowego kodu źródłowego. Dlatego kod musi zostać przekształcony w taki sposób, aby skutecznie oddawał jego właściwości strukturalne, syntaktyczne oraz semantyczne [2].

Głównym celem tworzenia reprezentacji kodu źródłowego jest umożliwienie modelom automatycznego wykrywania skomplikowanych wzorców w strukturach programów. Wybór konkretnej metody reprezentacji jest kluczowy, ponieważ determinuje on, jakie informacje o kodzie zostaną zachowane, a jakie utracone, co bezpośrednio wpływa na skuteczność algorytmu w danym zadaniu [3].

II. TAKSONOMIA I FORMALNE MODELOWANIE REPREZENTACJI KODU ŹRÓDŁOWEGO

W literaturze powtarza się dobrze utrwalona klasyfikacja reprezentacji kodu źródłowego ze względu na ich strukturę oraz poziom zawartej w nich informacji. Wyróżniamy reprezentacje leksykalne, syntaktyczne, semantyczne oraz hybrydowe i inne.

Posłużmy się przykładem kodu źródłowego napisanego w języku C w celu rozróżnienia charakterystyki każdej z wymienionych reprezentacji.

```
1 void foo() {  
2     int x = source();  
3     if(x < MAX) {  
4         int y = 2*x;  
5         sink(y);  
6     }  
7 }
```

Listing 1. Przykład kodu źródłowego do analizy reprezentacji [2]

A. Reprezentacje Leksykalne (Token-based)

Reprezentacja oparta na tokenach traktuje kod źródłowy jako wolny tekst lub płaską sekwencję znaków. Proces ten polega na konwersji kodu na listę tokenów, gdzie pojedynczym tokenem może być każde wyodrębnione słowo, a także każdy znak specjalny, taki jak nawias czy średnik.

Można wyróżnić kilka głównych podejść do podziału kodu na tokeny [1]:

- Tokenizacja na poziomie słów - token reprezentuje wyrazy lub znaki specjalne.
- Tokenizacja na poziomie znaków - token reprezentuje pojedyncze znaki.
- Tokenizacja na poziomie podsłów - kompromis między znakami, a słowami.
- Tokenizacja N-gram - token reprezentuje n-elementową sekwencję, np. pierwszy token to "void foo", a następny to "foo()", co dodaje informacje o kontekście sąsiadujących wyrazów.

```
1 ['void', 'foo', '(', ')', '{', 'int', 'x',  
2  '=', 'source', '(', ')', ';',  
3  'if', '(', 'x', '<', 'MAX', ')', '{', 'int',  
4  'y', '=', '2*x', ';',  
5  'sink', '(', 'y', ')', ';', '}', '}']
```

Listing 2. Reprezentacja kodu w formie ztokenizowanej na poziomie słów [2]

Aby modele Deep Learning mogły uczyć się z reprezentacji leksykalnej, tokeny są przekształcane na numeryczne wektory (embeddings) za pomocą statystycznych modeli językowych, takich jak word2vec lub doc2vec. Każdy token reprezentowany jest w ten sposób jako wektor liczb rzeczywistych [2].

B. Reprezentacje Syntaktyczne (Tree-based)

Reprezentacja syntaktyczna modeluje kod poprzez uwzględnienie jego hierarchicznej struktury oraz reguł gramatyki użytego języka programowania.

Najpopularniejszą strukturą w tym podejściu jest drzewo AST (Abstract Syntax Tree). Zamiast przetwarzać kod jako płaski ciąg znaków, AST konwertuje go w drzewo, w którym węzły reprezentują konkretne konstrukcje języka, takie jak wyrażenia, instrukcje, deklaracje [1]. Rysunek 1 przedstawia przykładową reprezentację kodu źródłowego w formie drzewa AST.

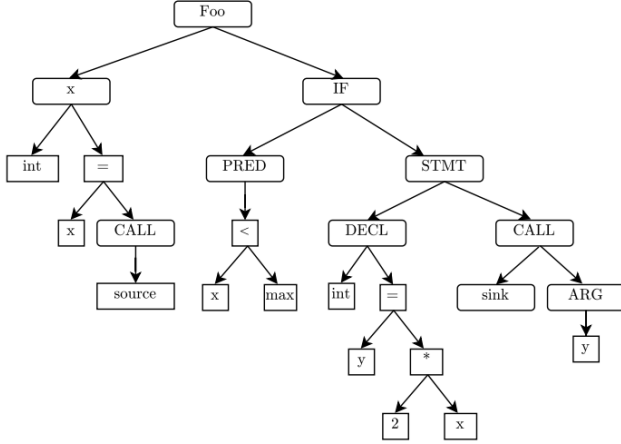


Fig. 1. Reprezentacja kodu bazująca na strukturze drzewiastej [2]

C. Reprezentacje Semantyczne (Graph-based)

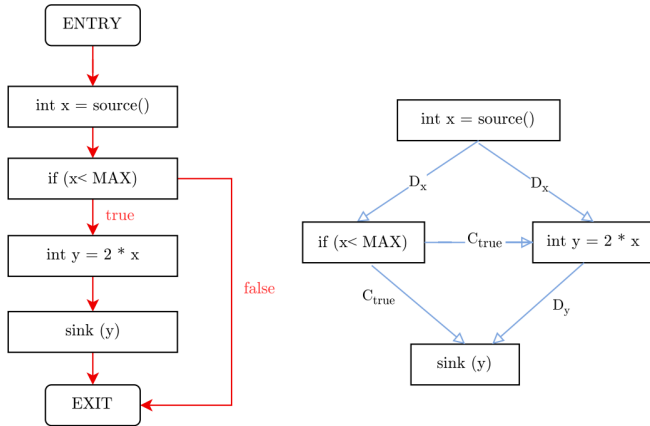


Fig. 2. Reprezentacja kodu bazująca na grafach przepływu CFG i PDG [2]

D. Reprezentacje Hybrydowe i Pośrednie

III. ZASTOSOWANIE REPREZENTACJI KODU ŹRÓDŁOWEGO W PROBLEMACH INŻYNIERII OPROGRAMOWANIA

tutaj o taskach SE w jakich stosuje się reprezentację kodu źródłowego czyli wykrywanie podatności, podsumowywanie

kodu, detekcja klonowania itp.

IV. ANALIZA SKUTECZNOŚCI REPREZENTACJI KODU ŹRÓDŁOWEGO W PROBLEMACH INŻYNIERII OPROGRAMOWANIA

Jakieś papiery z danymi o tym gdzie dana reprezentacja się najlepiej spisuje, ewentualnie zrobić swoją implementację i pokazać wyniki.

V. PODSUMOWANIE

Wnioski dotyczące wyboru optymalnej reprezentacji w zależności od dostępnych zasobów i specyfiki problemu.

BIBLIOGRAFIA

- [1] Shanmugasundaram, D., Arivukkarasu, P., Chen, H., & Cai, H. (2025). Deep Learning Representations of Programs: A Systematic Literature Review. *ACM Computing Surveys*, 58(5), 1-37.
- [2] Samoaa, H. P., Bayram, F., Salza, P., & Leitner, P. (2022). A systematic mapping study of source code representation for deep learning in software engineering. *IET Software*, 16(4), 351-385.
- [3] Siow, J. K., Liu, S., Xie, X., Meng, G., & Liu, Y. (2022, March). Learning program semantics with code representations: An empirical study. In *2022 IEEE international conference on software analysis, evolution and reengineering (SANER)* (pp. 554-565). IEEE.
- [4] Brauckmann, A., Goens, A., Ertel, S., & Castrillon, J. (2020, February). Compiler-based graph representations for deep learning models of code. In *Proceedings of the 29th International Conference on Compiler Construction* (pp. 201-211).
- [5] Casey, B., Santos, J. C., & Perry, G. (2025). A survey of source code representations for machine learning-based cybersecurity tasks. *ACM Computing Surveys*, 57(8), 1-41.